

Logbook — Projet Machine Learning

Estimation du prix des plaques d'immatriculation spéciales en Malaisie

Journal de bord — mis à jour à chaque étape du projet

23 mars 2026

Table des matières

1	Contexte et objectif du projet	3
1.1	Le marché des plaques spéciales en Malaisie	3
1.2	Objectif du projet	3
2	Étape 1 — Exploration initiale de la base de données	3
2.1	Description de la source	3
2.2	Analyse exploratoire (EDA)	3
2.3	Observations clés	5
3	Étape 2 — Nettoyage des données	6
3.1	Stratégie de nettoyage	6
3.2	Opérations effectuées	6
3.3	Le problème des doublons	6
3.4	Résultat final	7
3.5	Traitement des outliers de prix	7
4	Étape 3 — Split train / test	7
5	Étape 4 — Feature Engineering	8
5.1	Pourquoi le feature engineering est l'étape centrale	8
5.2	Features de structure	8
5.3	Features de numérologie et culture malaisienne	8
5.4	Features de patterns visuels	9
5.5	Features de rareté	9
5.6	Validation des features sur la base réelle	9
5.7	Features retirées et justification	9
5.8	Nuances importantes	10
6	Critique de la base de données	10
6.1	Limites identifiées	10
7	Étape 5 — Choix du modèle	11

7.1	Caractéristiques du problème	11
7.2	Analyse des candidats	11
7.3	Décision et justification	12
7.4	Transformation de la variable cible	12
8	Étape 6 — Entraînement et évaluation des modèles	12
8.1	Pipeline d'entraînement	12
8.2	Résultats	13
8.3	Analyse des résultats	13
8.4	Visualisations	14
9	Étape 7 — Amélioration du modèle (V2)	14
9.1	Motivations	14
9.2	Nouvelles features d'interaction	15
9.3	Résultats V2	15
9.4	Analyse	15
9.5	Modèle final retenu	16
9.6	Visualisations V2	16
10	Prochaines étapes	17
11	Étape 8 — Interface de prédiction	17
11.1	Description	17
11.2	Exemples de prédictions	17

1 Contexte et objectif du projet

1.1 Le marché des plaques spéciales en Malaisie

En Malaisie, les plaques d'immatriculation ne sont pas simplement des identifiants administratifs. Il existe un marché secondaire très actif où certaines plaques se vendent à des prix considérables, parfois plusieurs millions de ringgits malaisiens (RM). Ce phénomène est profondément ancré dans la culture locale, notamment sous l'influence de la communauté chinoise malaisienne qui accorde une grande importance à la numérologie.

Les critères qui rendent une plaque précieuse sont multiples : la rareté du numéro (un seul chiffre est bien plus rare que quatre), les patterns visuels (palindromes, chiffres répétés), et surtout la symbolique culturelle des chiffres. Le 8 est le chiffre le plus valorisé car il sonne comme “prospérité” en cantonais. Le 4, à l'inverse, sonne comme “mort” et est généralement évité. Ces plaques sont vendues aux enchères ou à prix fixe sur des plateformes spécialisées.

1.2 Objectif du projet

L'objectif est de construire un modèle de machine learning capable d'estimer le prix d'une plaque d'immatriculation malaisienne à partir de son texte brut uniquement. Le cas d'usage concret est simple : on croise une plaque dans la rue, on la note ou on la photographie, et on veut savoir combien elle vaut approximativement sur le marché.

Ce projet est un problème de **régression supervisée** : à partir de features extraites du texte de la plaque, on prédit une valeur numérique continue (le prix en RM).

2 Étape 1 — Exploration initiale de la base de données

2.1 Description de la source

La base de données initiale est un fichier CSV (`licence_plate_price.csv`) contenant deux colonnes :

- `plate` : le texte brut de la plaque (ex : WMA 4, EV 8888, QS 6 W)
- `price` : le prix au format texte tel qu'affiché sur le site source (ex : RM 3,800)

Les données ont été collectées par web scraping depuis le site **Motor Trader** (<https://www.motortrader.com.my>), une plateforme malaisienne spécialisée dans la vente de véhicules et de plaques d'immatriculation spéciales. Le fichier contient **3 420 entrées** à l'état brut.

2.2 Analyse exploratoire (EDA)

Un script d'exploration complet a été développé (`explore_data.py`) pour analyser la qualité des données avant tout nettoyage. Ce script génère un rapport visuel multi-panneaux incluant une heatmap des valeurs manquantes, la distribution des prix, et une répartition par tranches de prix.

Métrique	Valeur
Entrées totales	3 420
Valeurs NaN (colonne <code>plate</code>)	0
Valeurs NaN (colonne <code>price</code>)	0
Prix invalides (format POA)	107
Plaques mal formées	5
Prix minimum (entrées valides)	RM 500
Prix maximum (entrées valides)	RM 1 880 000
Prix médian	RM 5 800
Prix moyen	RM 25 806

TABLE 1 – Statistiques initiales de la base de données

La figure 1 présente la répartition des entrées valides et invalides, ainsi que le détail des suppressions effectuées lors du nettoyage.

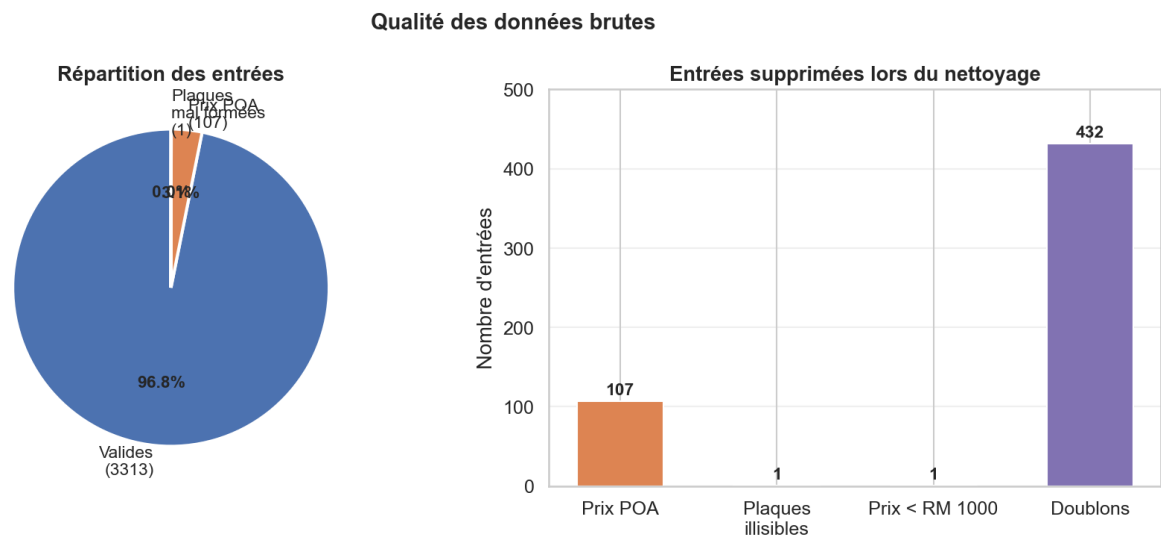


FIGURE 1 – Qualité des données brutes : répartition valides/invalides et détail des suppressions

La figure 2 montre la distribution des prix en échelle logarithmique. Cette représentation est la plus informative car elle révèle la structure réelle de la distribution, masquée en échelle linéaire par les quelques plaques à prix extrêmes.

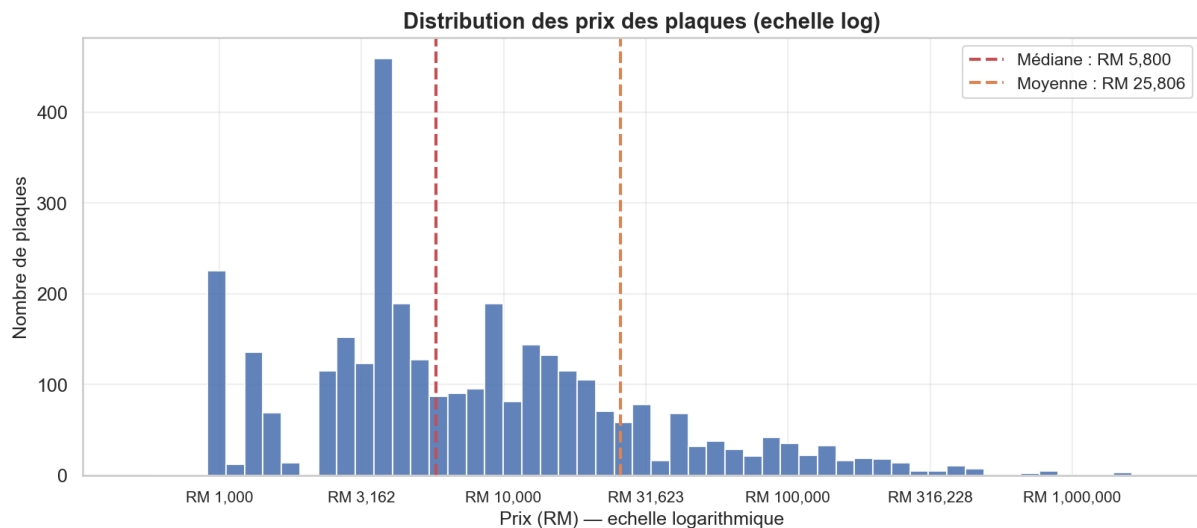


FIGURE 2 – Distribution des prix en échelle logarithmique — médiane et moyenne

La figure 3 détaille la répartition des plaques par tranche de prix, ce qui permet de visualiser concrètement la concentration du marché.

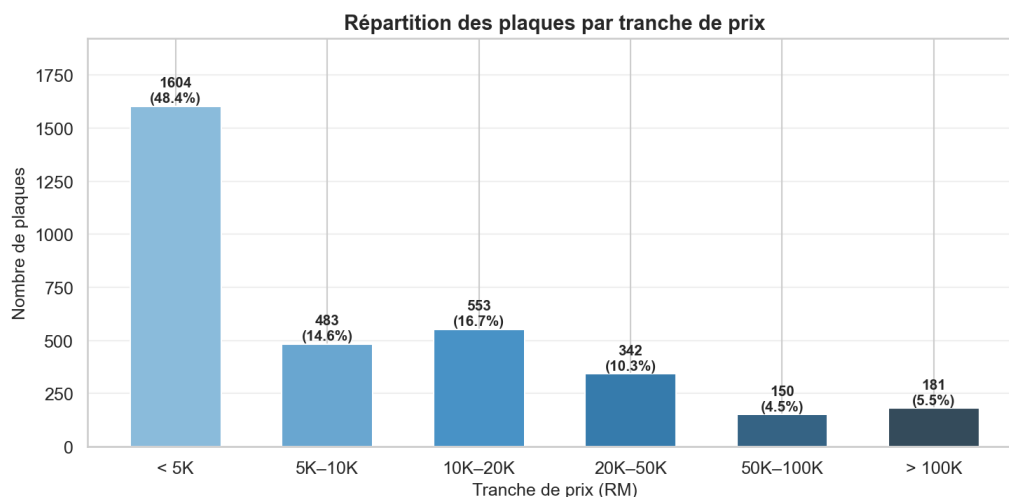


FIGURE 3 – Répartition des plaques par tranche de prix (RM)

2.3 Observations clés

Distribution des prix très asymétrique. La médiane (RM 5 800) est très éloignée de la moyenne (RM 25 806). Cela signifie que la grande majorité des plaques se vendent entre RM 2 000 et RM 10 000, mais quelques plaques extrêmement rares tirent la moyenne vers le haut. Cette asymétrie est visible sur l'histogramme : la distribution est fortement étalée vers la droite (*right-skewed*). En échelle logarithmique, la distribution devient beaucoup plus symétrique, ce qui suggère qu'une transformation log du prix pourrait améliorer les performances du modèle.

Absence de NaN. Aucune valeur manquante au sens strict (NaN) n'a été détectée. Les problèmes de qualité sont d'une autre nature : des prix non numériques (POA) et des plaques avec des caractères invalides.

Prix POA (Price On Application). 107 plaques ont un prix affiché comme “POA”, c’est-à-dire “prix sur demande”. Ces plaques ne peuvent pas être utilisées pour l’entraînement car elles n’ont pas de valeur numérique. Il est probable que ces plaques soient parmi les plus rares et les plus chères du marché, ce qui introduit un biais dans notre dataset (voir section 6).

3 Étape 2 — Nettoyage des données

3.1 Stratégie de nettoyage

Le nettoyage a été réalisé dans `clean_data.py`. La philosophie adoptée est de **supprimer plutôt que d’imputer** : avec seulement deux colonnes (plaque et prix), il n’y a pas de valeur à imputer — une entrée sans prix valide ou avec une plaque illisible est simplement inutilisable.

3.2 Opérations effectuées

1. Suppression des prix POA. Les 107 entrées avec `price = "POA"` ont été supprimées. Ces entrées représentent 3,1% du dataset. Elles ne peuvent pas contribuer à l’entraînement d’un modèle de régression.

2. Nettoyage des plaques mal formées. Cinq plaques présentaient des problèmes de format :

- Caractères spéciaux illisibles : `VP? 9` — supprimée (1 cas)
- Alias entre parenthèses : `JAB 4 (VIP 3)`, `CCG4 (CCC4)`, etc. — nettoyées par regex pour ne garder que la plaque principale (4 cas)
- Minuscules : `bqb 666` — converties en majuscules

3. Normalisation des espaces. Deux plaques présentaient des espaces multiples (`VKU 8228`, `VMJ 8228`). Ces espaces ont été normalisés à un espace simple.

4. Suppression du prix anormalement bas. Une seule entrée avait un prix inférieur à RM 1 000 : `JYV1103` à RM 500. Ce prix est anormalement bas par rapport au reste du marché (le minimum habituel est autour de RM 1 000–2 000). Cette entrée a été supprimée comme probable erreur de saisie.

5. Suppression des doublons. 432 doublons ont été détectés et supprimés (conservation de la première occurrence). Ce nombre élevé — 12,6% du dataset — est significatif et mérite une explication détaillée (voir ci-dessous).

3.3 Le problème des doublons

432 doublons représentent une proportion importante du dataset. Après analyse, il apparaît que la source de données liste plusieurs fois les mêmes plaques, probablement parce que les données ont été collectées à plusieurs reprises depuis le même site sans déduplication entre les collectes. Les doublons sont des entrées strictement identiques (même plaque,

même prix), ce qui confirme qu’il s’agit d’un artefact de collecte et non de vraies plaques différentes avec le même numéro.

Ce point est important pour interpréter la taille réelle du dataset : les 3 420 entrées initiales ne représentent en réalité que 2 879 plaques uniques.

3.4 Résultat final

Opération	Entrées supprimées
Prix POA	107
Plaques illisibles	1
Prix < RM 1 000	1
Doublons	432
Total supprimé	541
Entrées finales	2 879

TABLE 2 – Bilan du nettoyage

Le fichier nettoyé `licence_plate_price_cleaned.csv` contient deux colonnes : `plate` (texte normalisé en majuscules) et `price` (float en RM).

3.5 Traitement des outliers de prix

Une analyse IQR ($\times 3$) identifie 280 entrées au-dessus de RM 57 300 comme outliers statistiques. Ces entrées n’ont **pas été supprimées**. En effet, dans ce contexte, un prix de RM 1 880 000 pour la plaque KH1 n’est pas une erreur — c’est une plaque single digit avec un préfixe très court, légitimement l’une des plus rares du marché. Supprimer ces entrées priverait le modèle d’apprendre les cas extrêmes qui sont précisément les plus intéressants à prédire.

4 Étape 3 — Split train / test

Le dataset nettoyé a été divisé en deux ensembles distincts avant toute modélisation, conformément aux bonnes pratiques de machine learning. Cette séparation est essentielle pour évaluer les performances du modèle sur des données qu’il n’a jamais vues pendant l’entraînement.

Ensemble	Entrées	Proportion
Train	2 303	80%
Test	576	20%
Total	2 879	100%

TABLE 3 – Répartition train/test

Le split a été effectué avec `shuffle=True` et `random_state=42` pour garantir la reproductibilité. Aucune stratification n’a été appliquée (la stratification est utile pour les problèmes de classification avec des classes déséquilibrées, mais moins pertinente pour la régression).

5 Étape 4 — Feature Engineering

5.1 Pourquoi le feature engineering est l’étape centrale

Le modèle ne peut pas travailler directement sur le texte brut "EV 8888". Il faut transformer cette chaîne de caractères en un vecteur de nombres qui capturent les caractéristiques pertinentes pour le prix. C’est l’étape de *feature engineering*, et dans ce projet elle est particulièrement importante car **toute l’information disponible est contenue dans le texte de la plaque**.

Contrairement à d’autres projets ML où les features sont déjà présentes dans le dataset (âge, revenu, etc.), ici nous devons les construire entièrement à partir de zéro en nous appuyant sur notre compréhension du marché malaisien.

Toutes les features sont implémentées dans `features.py`, un module réutilisable importé par `train.py` et `predict.py`.

5.2 Features de structure

Ces features décrivent la forme brute de la plaque.

- `prefix_len` : longueur du préfixe lettres. Une plaque comme EV 8888 a un préfixe de 2 lettres, VPA 6363 en a 3. Les séries à préfixe court sont plus anciennes et donc plus rares. C’est l’une des features les plus discriminantes.
- `num_digits` : nombre de chiffres dans la plaque. C’est probablement la feature la plus importante : les plaques à 1 chiffre (KH1, VR1) atteignent des prix de plusieurs centaines de milliers à plusieurs millions de RM, tandis que les plaques à 4 chiffres se vendent généralement entre RM 2000 et RM 10 000.
- `total_len` : longueur totale de la plaque sans espaces.
- `digits_value` : valeur numérique brute du numéro (ex : 4 pour WMA 4).

5.3 Features de numérologie et culture malaisienne

Ces features capturent les croyances culturelles qui influencent directement le marché.

- `lucky_count` et `lucky_ratio` : nombre et proportion de chiffres chanceux (8, 9, 6, 1). Le 8 est le plus valorisé (“prospérité” en cantonais), suivi du 9 (“longévité”), du 6 (“fluidité”) et du 1 (“premier”, “unique”).
- `unlucky_count` : nombre de 4. Le 4 sonne comme “mort” (“sì”) en cantonais et mandarin. En théorie, sa présence devrait faire baisser le prix. En pratique, l’analyse montre que ce n’est pas toujours le cas (voir section 5.8).
- `digit_sum` : somme des chiffres, utilisée en numérologie.
- `ends_8`, `starts_8` : indicateurs booléens pour la position du 8.
- `contains_888`, `contains_8888` : la séquence 888 est particulièrement valorisée. DFB 888 se vend RM 138 000, JT 888 RM 118 800.

5.4 Features de patterns visuels

Ces features capturent l'esthétique et la mémorabilité de la plaque.

- **all_same** : tous les chiffres sont identiques (ex : 8888, 1111, 9999). Ces plaques sont très recherchées. Y 9999 se vend RM 218 800.
- **is_palindrome** : le numéro se lit pareil dans les deux sens (ex : 1221, 8558, 9999). UU 88 à RM 588 000 est à la fois palindrome et all_same.
- **is_abab** : pattern de répétition ABAB (ex : 1212, 8989, 1313). IM 1313 se vend RM 68 000.
- **max_repeat** : longueur de la plus longue séquence de chiffres identiques consécutifs. Permet de capturer des cas comme 888 dans SJR 9888 (max_repeat=3).

5.5 Features de rareté

- **is_single** : numéro à 1 chiffre (1–9). Les plaques les plus rares et les plus chères du marché.
- **is_double** : numéro à 2 chiffres (10–99).
- **is_round** : multiple de 100 (ex : 100, 400, 1000). Ces numéros “ronds” ont une certaine valeur esthétique.
- **is_short_prefix** : préfixe de 1 ou 2 lettres, indicateur de série ancienne.

5.6 Validation des features sur la base réelle

Chaque feature a été vérifiée en cherchant des exemples réels dans la base et en comparant avec les prix observés. Le tableau suivant présente quelques exemples représentatifs :

Plaque	Prix réel	Features actives
KH1	RM 1 880 000	is_single, prefix_len=2, all_same
T10	RM 988 000	is_double, prefix_len=1
UU 88	RM 588 000	is_double, all_same, is_palindrome, ends_8
DFB 888	RM 138 000	contains_888, max_repeat=3, lucky_count=3
Y 9999	RM 218 800	all_same, is_palindrome, lucky_count=4, prefix_len=1
IM 1313	RM 68 000	is_abab, lucky_count=2
PSD 999	RM 21 300	all_same, is_palindrome, lucky_count=3, num_digits=3
VPA 6363	RM 3 800	is_abab, lucky_count=2, prefix_len=3
VKY 2797	RM 1 000	aucun pattern, unlucky_count=0, prefix_len=3

TABLE 4 – Exemples de validation des features sur la base réelle

La cohérence est bonne : les features discriminent bien les plaques rares des communes.

5.7 Features retirées et justification

is_sequential et is_rev_seq. Ces deux features détectaient les suites croissantes (ex : 1234, 5678) et décroissantes (ex : 9876, 4321). Elles ont été retirées pour une raison statistique simple : la base ne contient que **8 cas séquentiels** et **4 cas décroissants**,

soit moins de 0,5% du dataset. Avec si peu d'exemples, le modèle ne peut pas apprendre un signal fiable — ces features introduiraient du bruit plutôt que de l'information.

De plus, l'analyse montre que le prix d'une plaque séquentielle dépend surtout d'autres features déjà présentes : R 8765 vaut RM 26 800 principalement à cause de son préfixe à 1 lettre (`prefix_len=1`), pas parce que c'est une suite décroissante. La feature `is_rev_seq` n'apporterait donc pas d'information supplémentaire dans ce cas.

À améliorer : avec une base de données plus large incluant davantage de plaques séquentielles, ces features pourraient être réintégrées.

5.8 Nuances importantes

Le chiffre 4 ne pénalise pas toujours. La feature `unlucky_count` repose sur l'hypothèse que le 4 fait baisser le prix. L'analyse sur la base réelle montre que ce n'est pas systématique : DT 4 se vend RM 259 800 et A 44 A RM 218 800 malgré la présence de 4. Quand la plaque est courte et rare, le marché valorise la rareté avant la numérologie. La feature est conservée mais le modèle devra apprendre cette interaction avec d'autres features (notamment `num_digits` et `prefix_len`).

Redondance partielle de `all_same`. Pour les plaques à 1 chiffre (`is_single=1`), la feature `all_same` est mécaniquement vraie et donc redondante. Elle apporte de l'information uniquement pour les plaques à 3–4 chiffres (ex : 999, 8888). Un modèle comme Gradient Boosting gère bien cette redondance partielle.

6 Critique de la base de données

6.1 Limites identifiées

1. Taille modeste. 2 879 entrées après nettoyage est une taille limitée pour un problème de régression avec une variable cible couvrant un ratio de 1 à 1 880 (de RM 1 000 à RM 1 880 000). Les cas extrêmes — plaques très chères et très communes — sont sous-représentés, ce qui rend difficile l'apprentissage des frontières de prix.

2. Doublons à la source. 432 doublons (12,6% du dataset initial) indiquent une collecte non rigoureuse. La base provient probablement d'un scraping répété du même site sans déduplication entre les collectes. Cela signifie que le dataset réel est encore plus petit qu'il n'y paraît.

3. Sous-représentation de certains patterns. Les plaques séquentielles (8 cas) et décroissantes (4 cas) sont trop rares pour être apprises correctement. D'autres patterns culturels spécifiques pourraient également être sous-représentés. Une base plus large permettrait d'inclure ces features.

4. Absence de contexte temporel. Les prix des plaques évoluent avec le marché (enchères, tendances économiques, popularité de certains chiffres). La base ne contient aucune date, il est donc impossible de savoir si les prix sont récents ou anciens, ni de modéliser l'évolution temporelle des prix.

5. Biais lié aux prix POA. 107 plaques ont un prix “sur demande” (POA) et ont dû être exclues. Ces plaques sont probablement parmi les plus rares et les plus chères du marché — leur exclusion crée un biais vers le bas dans les estimations pour les plaques très rares. Le modèle risque de sous-estimer les prix des plaques les plus exceptionnelles.

6. Absence d’information contextuelle. Le prix d’une plaque peut dépendre de facteurs non capturés dans notre dataset : l’état de la plaque (neuve vs. déjà immatriculée), la région d’émission, la popularité du préfixe dans une communauté particulière, ou encore la date de l’enchère. Ces informations ne sont pas disponibles.

7 Étape 5 — Choix du modèle

7.1 Caractéristiques du problème

Avant de choisir un modèle, il faut caractériser précisément le problème pour identifier les contraintes auxquelles le modèle devra répondre :

- **Type de problème** : régression supervisée — on prédit une valeur numérique continue (le prix en RM).
- **Taille du dataset** : 2 303 exemples d’entraînement. C’est un dataset de taille modeste.
- **Nature des features** : mixte — features booléennes (0/1), entières (nombre de chiffres, longueur du préfixe), et continues (ratio de chiffres chanceux, valeur numérique). Aucune feature catégorielle ordinale complexe.
- **Distribution de la cible** : très asymétrique (right-skewed). La majorité des prix se concentre entre RM 2 000 et RM 10 000, mais quelques plaques atteignent plusieurs centaines de milliers voire des millions de RM. Cette asymétrie est un défi pour les modèles qui minimisent l’erreur quadratique moyenne (MSE), car ils seront fortement influencés par les valeurs extrêmes.
- **Interactions non-linéaires** : les features interagissent de façon complexe. Par exemple, la présence du chiffre 4 (`unlucky_count`) ne pénalise le prix que si la plaque n’est pas par ailleurs très rare (`num_digits` élevé, `prefix_len` long). Ces interactions ne peuvent pas être capturées par un modèle linéaire.

7.2 Analyse des candidats

Régression linéaire. Inadaptée. Les relations entre les features et le prix ne sont pas linéaires — un single digit ne vaut pas simplement “`prefix_len` × coefficient”. La régression linéaire ne peut pas capturer les interactions entre features ni les effets multiplicatifs. Éliminée d’emblée.

Support Vector Regression (SVR). Peu adapté. Sensible à l’échelle des features, nécessite une normalisation soigneuse, et ne gère pas bien les interactions complexes sur des données tabulaires avec de nombreuses features binaires. Les performances sur ce type de problème sont généralement inférieures aux méthodes ensemblistes.

Réseau de neurones (MLP). Surdimensionné pour ce problème. Avec seulement 2 303 exemples d’entraînement et 20 features, un réseau de neurones risque fortement le surapprentissage. De plus, les réseaux de neurones ne sont pas les plus efficaces sur des données tabulaires de petite taille — les méthodes ensemblistes les surpassent généralement dans ce contexte.

Random Forest Regressor. Bon candidat. Robuste aux outliers, gère naturellement les interactions entre features, ne nécessite pas de normalisation, et est peu sensible aux hyperparamètres. Cependant, le Random Forest a une limitation importante pour ce problème : il prédit en faisant la **moyenne des prédictions de ses arbres**, ce qui le pousse à régresser vers la moyenne et à sous-estimer les valeurs extrêmes. Or, les plaques les plus chères sont précisément les plus intéressantes à prédire correctement.

Gradient Boosting Regressor (GBR). Meilleur choix pour ce problème. Contrairement au Random Forest qui construit ses arbres en parallèle de façon indépendante, le Gradient Boosting construit les arbres **séquentiellement**, chaque arbre corrigeant les erreurs du précédent. Cette approche lui permet de mieux capturer les cas difficiles, notamment les valeurs extrêmes. Il gère très bien les features avec peu de valeurs uniques (booléennes) et les interactions non-linéaires complexes.

7.3 Décision et justification

Modèle principal : Gradient Boosting Regressor. Modèle de référence (baseline) : Random Forest Regressor.

Le Random Forest servira de baseline pour évaluer le gain apporté par le Gradient Boosting. Si les deux modèles donnent des performances similaires, cela indiquera que le problème est bien capturé par les features et que le choix du modèle est secondaire.

7.4 Transformation de la variable cible

La distribution des prix étant fortement asymétrique, le modèle sera entraîné sur **log(prix)** plutôt que sur le prix brut. Cette transformation présente deux avantages :

1. Elle rend la distribution de la cible plus symétrique, ce qui facilite l’apprentissage et réduit l’influence disproportionnée des valeurs extrêmes sur la fonction de perte.
2. Elle est cohérente avec la perception humaine des prix : une erreur de RM 5 000 sur une plaque à RM 10 000 est bien plus grave qu’une erreur de RM 5 000 sur une plaque à RM 500 000. En travaillant en log, on minimise l’erreur relative plutôt que l’erreur absolue.

Les prédictions seront reconverties en RM (via $e^{\hat{y}}$) pour l’évaluation finale et l’interface utilisateur.

8 Étape 6 — Entraînement et évaluation des modèles

8.1 Pipeline d’entraînement

L’entraînement est réalisé dans `train.py`. Le pipeline complet est le suivant :

1. Chargement de `train.csv` et `test.csv`
2. Extraction des features via `features.py` (`build_feature_matrix`)
3. Encodage du préfixe lettres par `LabelEncoder`
4. Transformation de la cible : $y = \log(\text{prix})$
5. Entraînement des deux modèles
6. Évaluation sur le jeu de test (reconversion $\hat{y} \rightarrow e^{\hat{y}}$ pour les métriques en RM)
7. Sauvegarde du meilleur modèle dans `models/best_model.pkl`

8.2 Résultats

Modèle	MAE (RM)	R ² (log)	MAPE
Gradient Boosting	8 259	0.9180	31.1%
Random Forest	5 906	0.9206	28.7%

TABLE 5 – Comparaison des performances sur le jeu de test

8.3 Analyse des résultats

Le Random Forest surpasse le Gradient Boosting. Ce résultat est contre-intuitif par rapport à notre analyse préalable. Plusieurs explications sont possibles :

- **Taille du dataset** : avec seulement 2 303 exemples d’entraînement, le Gradient Boosting — qui est un algorithme plus complexe avec plus d’hyperparamètres — est plus difficile à optimiser et peut souffrir d’un léger surapprentissage. Le Random Forest, plus robuste par construction (bagging), se comporte mieux sur les petits datasets.
- **Structure des features** : la majorité des features sont binaires (0/1). Le Random Forest gère très efficacement ce type de features car chaque arbre peut créer des partitions nettes sur ces valeurs. Le Gradient Boosting, qui construit des arbres peu profonds (`max_depth=4`), peut avoir du mal à capturer toutes les interactions.

Interprétation des métriques.

- **R² = 0.92** : le modèle explique 92% de la variance des prix (en échelle log). C’est un très bon résultat pour un problème avec une distribution aussi étalée.
- **MAE = RM 5 906** : en moyenne, le modèle se trompe de RM 5 906. Rapporté à la médiane du marché (RM 5 800), cela représente une erreur d’environ 100% sur les plaques bon marché, mais une erreur bien plus faible en proportion sur les plaques chères.
- **MAPE = 28.7%** : l’erreur relative moyenne est de 28,7%. C’est acceptable pour un marché aussi hétérogène, mais indique que le modèle a du mal sur certaines tranches de prix.

Modèle retenu. Le **Random Forest** est retenu comme modèle final et sauvegardé dans `models/best_model.pkl`.

8.4 Visualisations

La figure 4 compare les prix prédits aux prix réels pour les deux modèles. Les points proches de la diagonale rouge indiquent de bonnes prédictions.

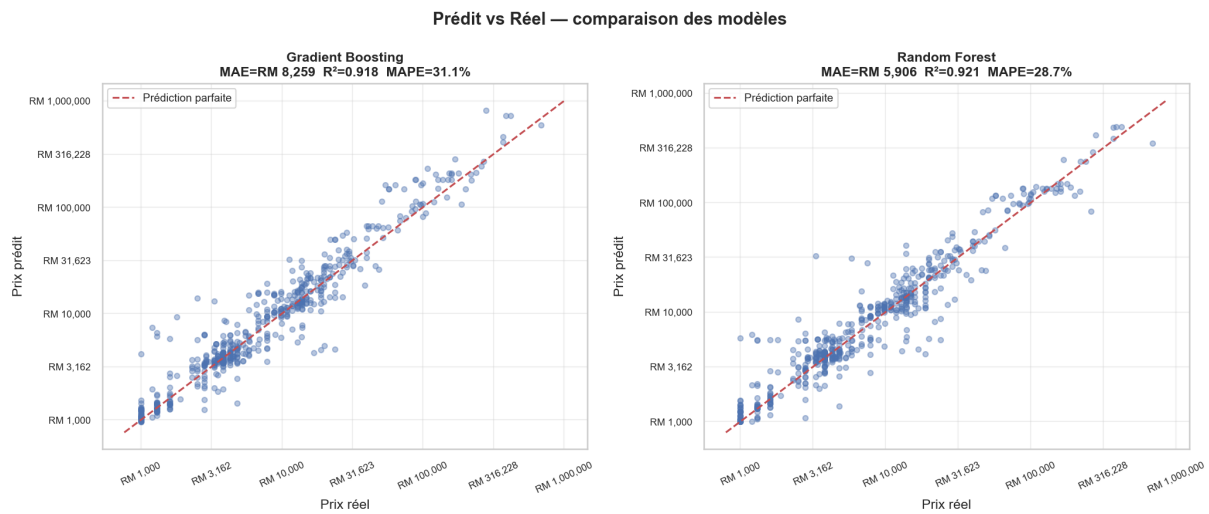


FIGURE 4 – Prix prédit vs prix réel — Gradient Boosting et Random Forest (échelle log)

La figure 5 présente les features les plus importantes selon le Random Forest.

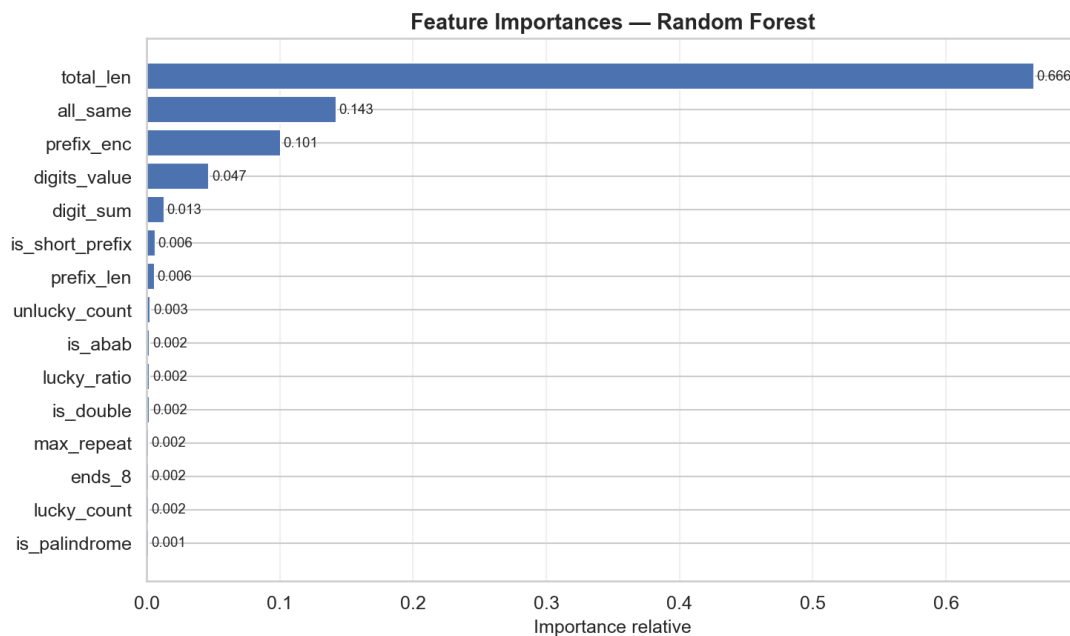


FIGURE 5 – Feature importances — Random Forest

9 Étape 7 — Amélioration du modèle (V2)

9.1 Motivations

Après les résultats de la V1 (Random Forest, MAE=RM 5 906, $R^2=0.9206$), trois pistes d'amélioration ont été explorées simultanément :

1. **Features d'interaction manuelle** : aider le modèle en créant explicitement les combinaisons de features les plus importantes.
2. **LightGBM** : implémentation optimisée du gradient boosting, plus efficace que le GBR de sklearn.
3. **Stacking** : combiner Random Forest et LightGBM avec un méta-modèle Ridge.

9.2 Nouvelles features d'interaction

Six features d'interaction ont été ajoutées au vecteur de features existant (21 features de base → 27 features au total) :

- **rarity_score** : $\frac{1}{\text{num_digits} \times \text{prefix_len}}$ — score de rareté combinée. Plus le numéro est court et le préfixe court, plus ce score est élevé.
- **same_and_lucky** : $\text{all_same} \times \text{lucky_count}$ — distingue 8888 (très chanceux) de 1111 (all_same mais peu chanceux).
- **single_short** : $\text{is_single} \times (\text{prefix_len} \leq 2)$ — capture l'effet exponentiel d'un single digit avec préfixe court.
- **palin_lucky** : $\text{is_palindrome} \times \text{lucky_count}$ — palindrome avec chiffres chanceux.
- **digits_x_repeat** : $\text{num_digits} \times \text{max_repeat}$ — capture les cas comme 8888 (4 chiffres, tous répétés).
- **unlucky_weighted** : $\text{unlucky_count} \times \text{num_digits}$ — la pénalité du 4 est pondérée par la longueur : sur une plaque longue, le 4 pénalise plus que sur une plaque courte et rare.

9.3 Résultats V2

Modèle	MAE (RM)	R ² (log)	MAPE
V1 — Random Forest (référence)	5 906	0.9206	28.7%
V2 — Random Forest	5 644	0.9149	30.9%
V2 — LightGBM	5 849	0.9127	32.9%
V2 — Stacking (RF + LightGBM)	5 605	0.9158	31.7%

TABLE 6 – Comparaison V1 vs V2

9.4 Analyse

Le stacking est le meilleur modèle V2 sur la MAE. Le stacking (RF + LightGBM avec méta-modèle Ridge) atteint une MAE de RM 5 605, soit une réduction de RM 301 par rapport à la V1. Les features d'interaction contribuent à cette amélioration en aidant le modèle à mieux capturer les effets combinés.

Trade-off MAE vs R² vs MAPE. Un résultat contre-intuitif apparaît : le R² de la V2 est légèrement inférieur à celui de la V1 (0.9158 vs 0.9206), et la MAPE est plus élevée (31.7% vs 28.7%). Cela signifie que le stacking réduit les grosses erreurs absolues (MAE)

mais est moins précis en proportion sur les plaques bon marché. Ce trade-off est acceptable pour notre cas d'usage : une erreur de RM 300 sur une plaque à RM 1 000 est plus gênante qu'une erreur de RM 5 000 sur une plaque à RM 500 000.

LightGBM décevant. LightGBM, pourtant réputé très performant sur les données tabulaires, ne surpasse pas le Random Forest ici. Cela confirme que la taille modeste du dataset (2 303 exemples) favorise les méthodes ensemblistes simples comme le Random Forest, moins sensibles aux hyperparamètres.

9.5 Modèle final retenu

Le **Stacking (RF + LightGBM)** est retenu comme modèle final V2, sauvegardé dans `train_v2/models/best_model_v2.pkl`.

9.6 Visualisations V2

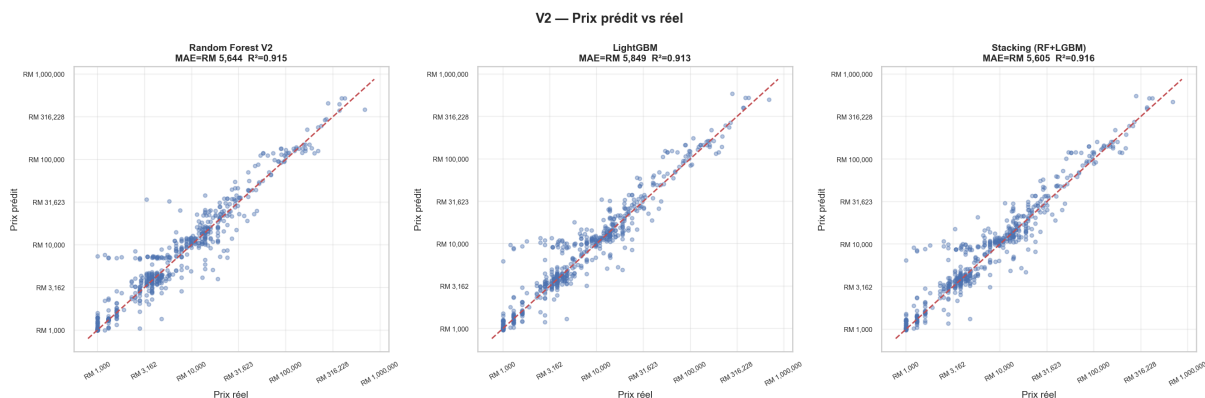


FIGURE 6 – V2 — Prix prédit vs réel pour les trois modèles

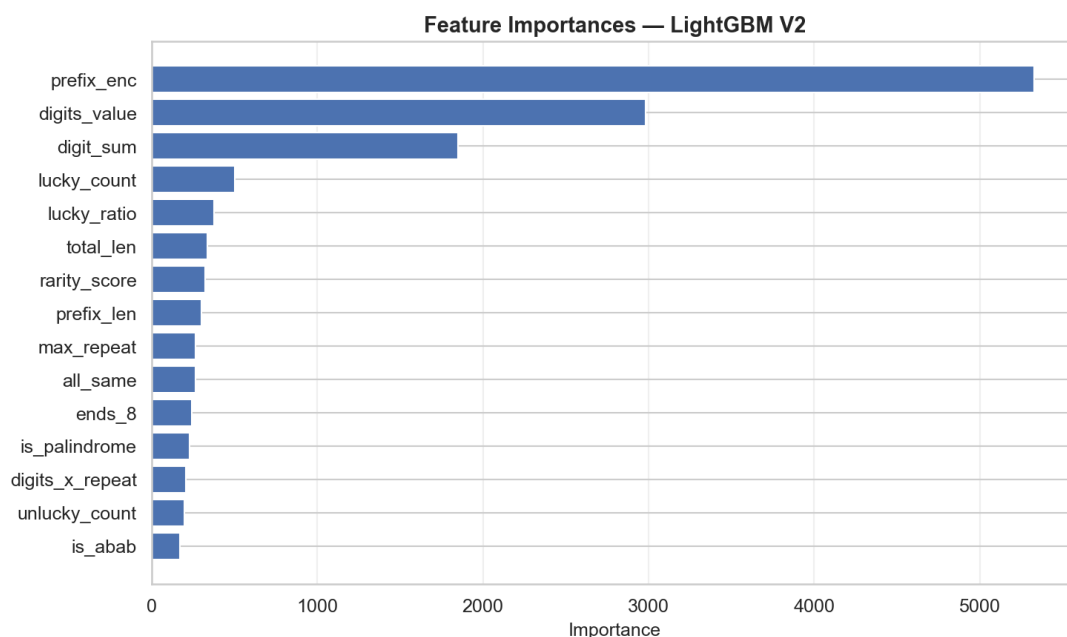


FIGURE 7 – V2 — Feature importances LightGBM (incluant les features d'interaction)

10 Prochaines étapes

Entraînement des modèles V1 (Gradient Boosting + Random Forest)

Évaluation sur le jeu de test (MAE, R^2 , MAPE)

Amélioration V2 (features d'interaction + LightGBM + Stacking)

Analyse des feature importances

11 Étape 8 — Interface de prédiction

11.1 Description

Le script `predict.py` constitue l'interface utilisateur finale du projet. Il charge le modèle V2 (Stacking RF+LightGBM) et permet d'estimer le prix de n'importe quelle plaque en ligne de commande.

Deux modes d'utilisation :

- Mode direct : `python predict.py "WMA 4"`
- Mode interactif : `python predict.py` (saisie en boucle)

Pour chaque plaque, le script affiche le prix estimé et une explication des facteurs qui influencent la valeur.

11.2 Exemples de prédictions

Plaque	Prix estimé	Prix réel	Facteurs principaux
KH 1	RM 1 448 048	RM 1 880 000	single digit, préfixe court
EV 8888	RM 56 791	RM 68 800	8888, all_same, préfixe court
VPA 6363	RM 3 088	RM 3 800	pattern ABAB
WMA 4	RM 52 770	—	single digit, pénalité 4

TABLE 7 – Exemples de prédictions du modèle final

Les estimations sont dans le bon ordre de grandeur. L'écart sur KH 1 (RM 432 000) s'explique par le fait que les plaques les plus extrêmes sont sous-représentées dans le dataset d'entraînement.

Dashboard Streamlit (optionnel)